

git下载与GitHub注册

https://git-scm.com/?utm_source=chatgpt.com

https://github.com/?utm_source=chatgpt.com

一、Git 和 GitHub 是什么

- **Git (工具)**：就像是单机游戏里的“存档管理器”。它在你的电脑本地运行，帮你记录代码的每一次修改、每一个版本。
- **GitHub (平台)**：就像是“游戏云端存档中心”。它是一个网络平台，让你把本地的存档传到云端，方便备份或者和队友一起联机（协同开发）。

二、下载安装 Git

1. 打开 Git 官网

[Git 下载页面](#)

下载：

64-bit Git for Windows Setup

三、安装 Git（重点）

安装时：

一路 Next 即可

只改一个地方：

PATH 环境变量（重点）

看到：

```
Adjusting your PATH environment
```

选择：

```
Git from the command line and also from 3rd-party software
```

意思：

直接把 Git 加入系统环境变量

这样：

- cmd 能用 git
- PowerShell 能用 git
- VSCode 能用 git
- 后面不用再手动配置环境变量

四、验证 Git 是否安装成功

安装完成后：

打开：

```
cmd
```

输入：

```
git --version
```

看到：

```
git version 2.xx.x
```

说明安装成功。

六、注册 GitHub

打开 GitHub

[GitHub 官网](#)

注册：

- 邮箱
- 用户名
- 密码

建议：

- 用户名用英文
- 后续会成为你的项目地址

例如：

```
https://github.com/用户名
```

七、配置 Git（第一次必须做）

打开：

- CMD
 - PowerShell
 - VSCode终端
- 都可以。

设置用户名

```
git config --global user.name "你的名字"
```

例如：

```
git config --global user.name "xiaoming"
```

设置邮箱（用你注册GitHub的邮箱）

```
git config --global user.email "你的邮箱"
```

例如：

```
git config --global user.email "123@qq.com"
```

查看配置（看看你的用户名和密码有没有配置错）

```
git config --list
```

八、配置 GitHub SSH（推荐）

现在 GitHub 基本不用密码推送了。

推荐 SSH。

生成 SSH Key



输入：

```
ssh-keygen -t ed25519 -C "你的GitHub邮箱"
```

例如：

```
ssh-keygen -t ed25519 -C "123@qq.com"
```

一路回车

生成位置：

C:\Users\用户名\.ssh

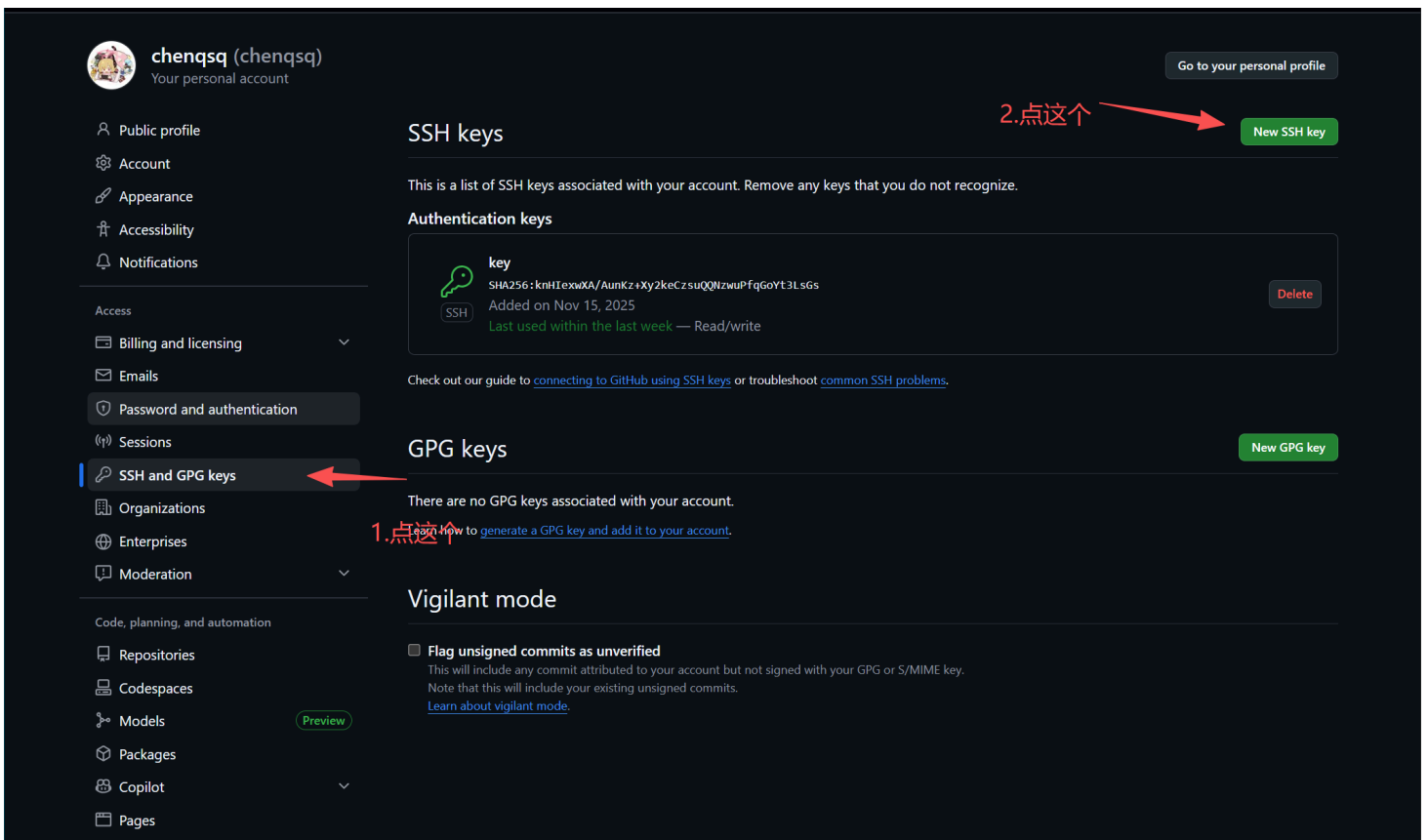
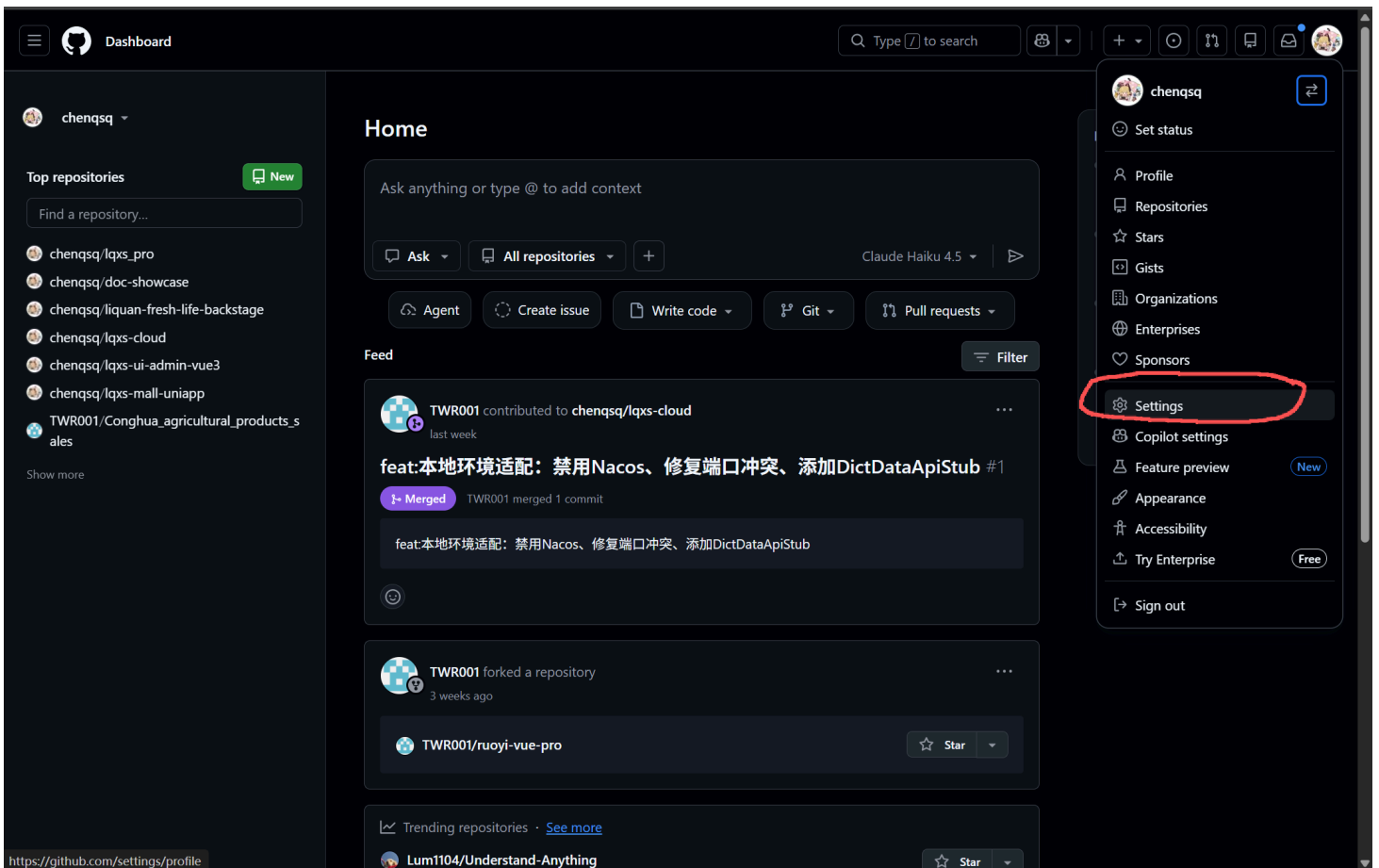
查看公钥



打开文件后复制全部内容。

九、添加 SSH Key 到 GitHub

打开设置：



把你刚刚复制到的ssh密钥粘贴到这里面

Add new SSH Key

Title

1.这里是写密钥名字，用英文名

Key type

Authentication Key

Key

Begins with 'ssh-rsa', 'ecdsa-sha2-nistp256', 'ecdsa-sha2-nistp384', 'ecdsa-sha2-nistp521', 'ssh-ed25519', 'sk-ecdsa-sha2-nistp256@openssh.com', or 'sk-ssh-ed25519@openssh.com'

2.这里粘贴密钥

Add SSH key

3.点确定添加

十、测试 SSH 是否成功

输入：

```
ssh -T git@github.com
```

第一次输入：

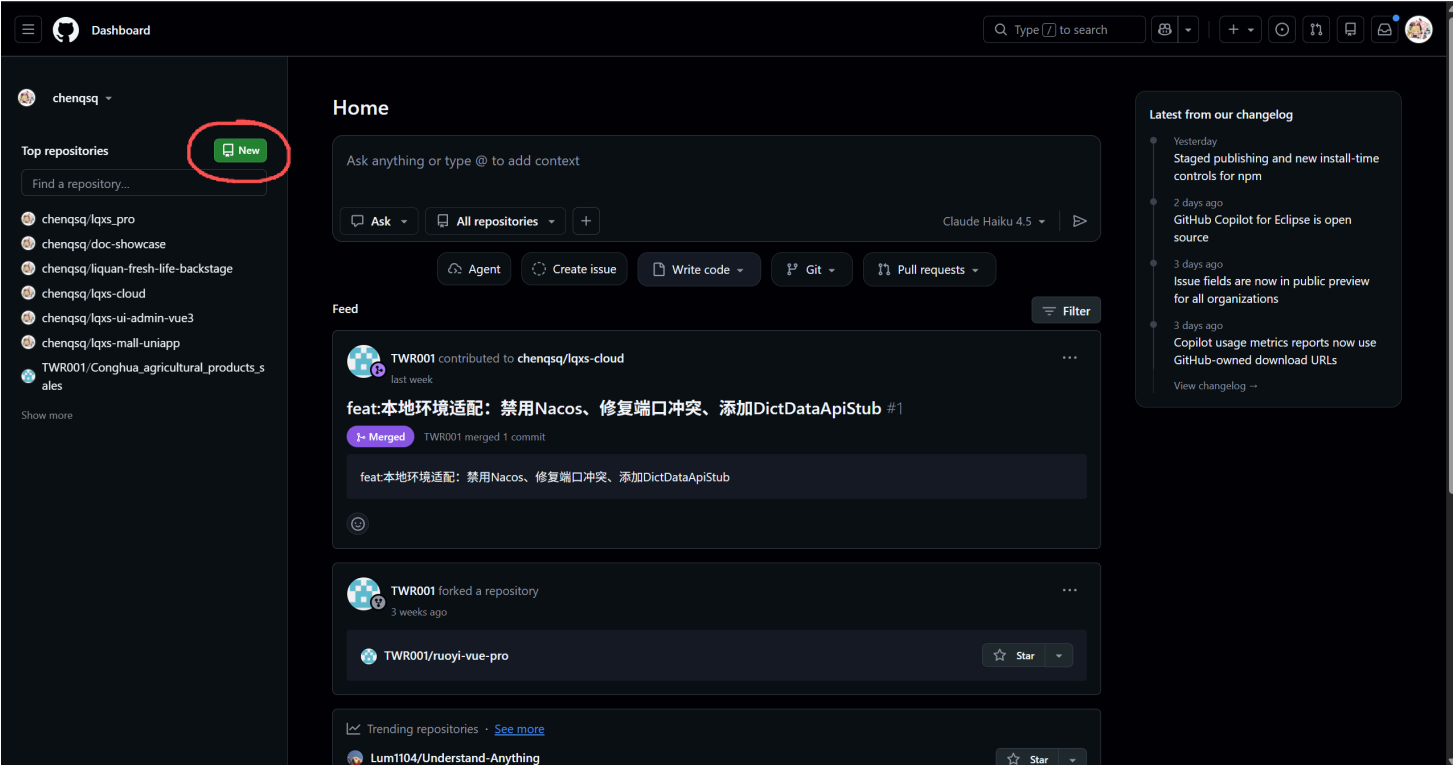
```
yes
```

成功后会看到：

```
You've successfully authenticated
```


十一、创建 GitHub 仓库

GitHub 首页：



Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

1 General

Owner *

 chenqsq ▾

Repository name *

/

1.写上你的代码仓库名字，用英文

Great repository names are short and memorable. How about [effective-waddle](#)?

Description

0 / 350 characters

2.仓库介绍，随便写点

2 Configuration

Choose visibility *

Choose who can see and commit to this repository



Public ▾

Add README

READMEs can be used as longer descriptions. [About READMEs](#)

3.把这个打成on

On

Off

Add .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

No .gitignore ▾

Add license

Licenses explain how others can use your code. [About licenses](#)

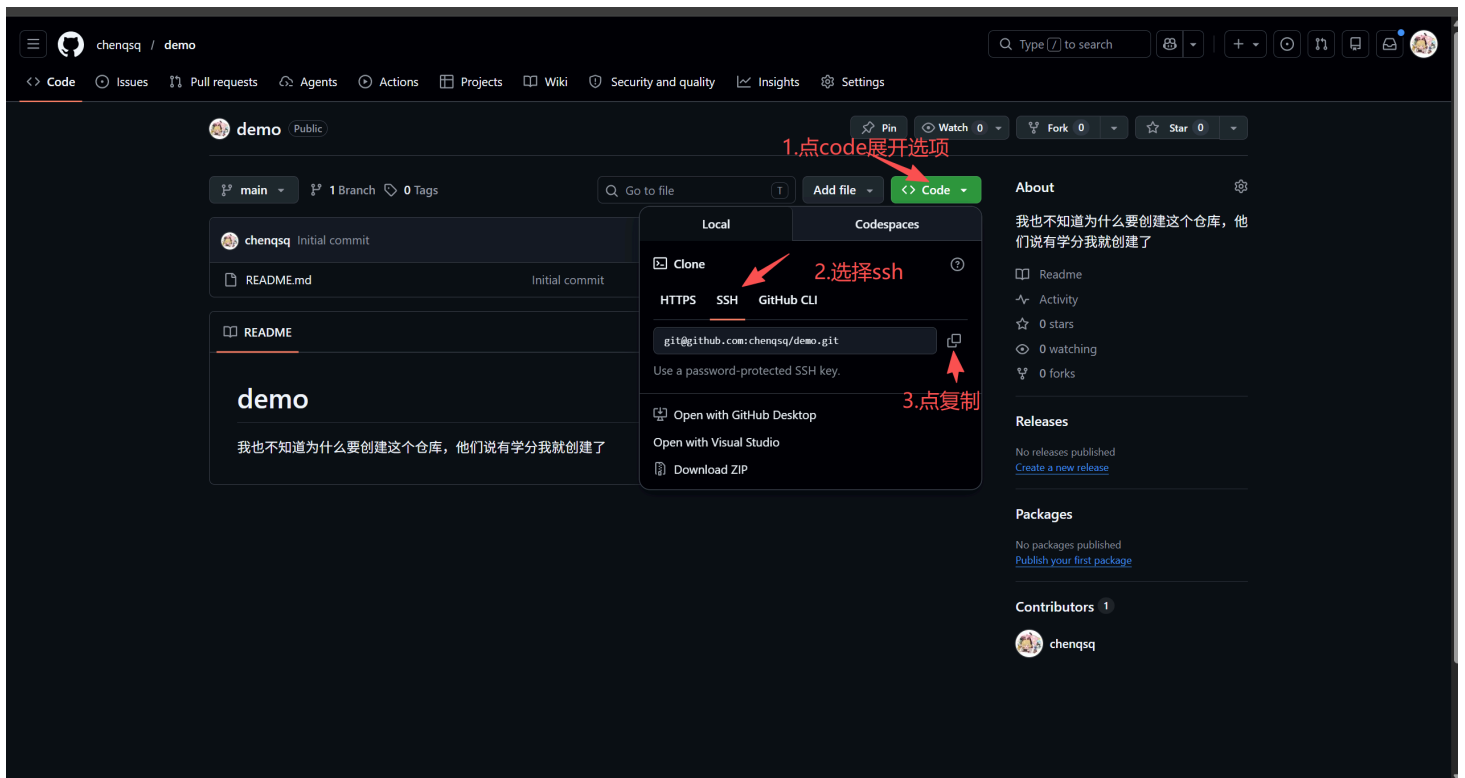
No license ▾

Create repository

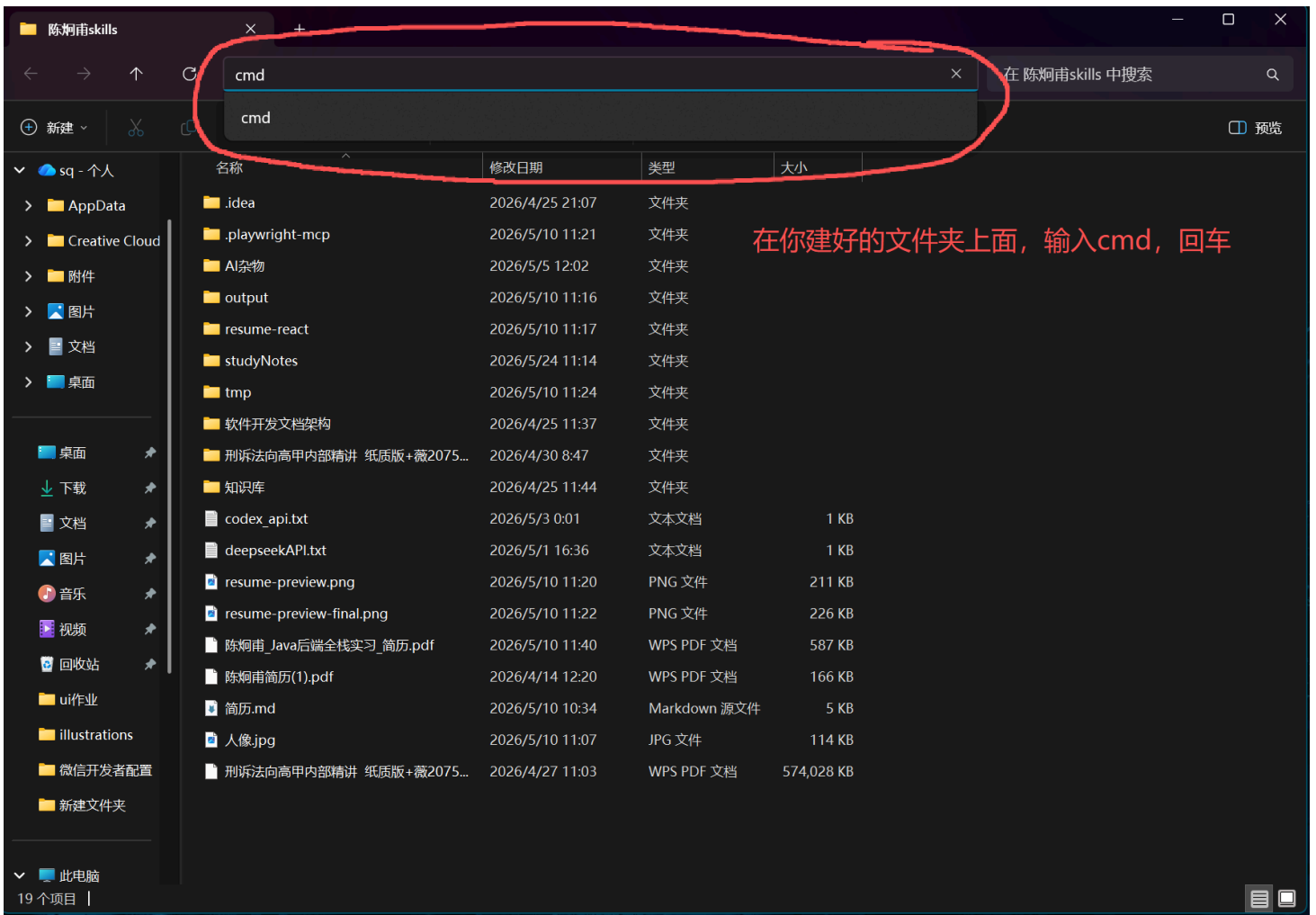
4.创建仓库

十二、克隆仓库到本地

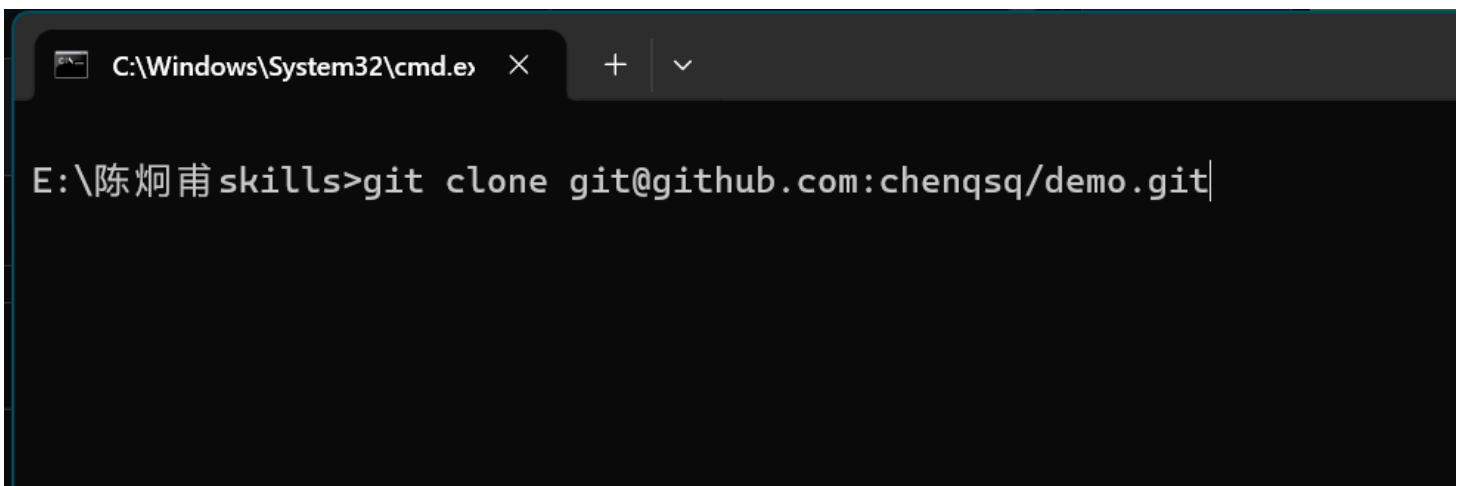
复制仓库 SSH 地址：



自己新建一个文件夹，然后打开终端：



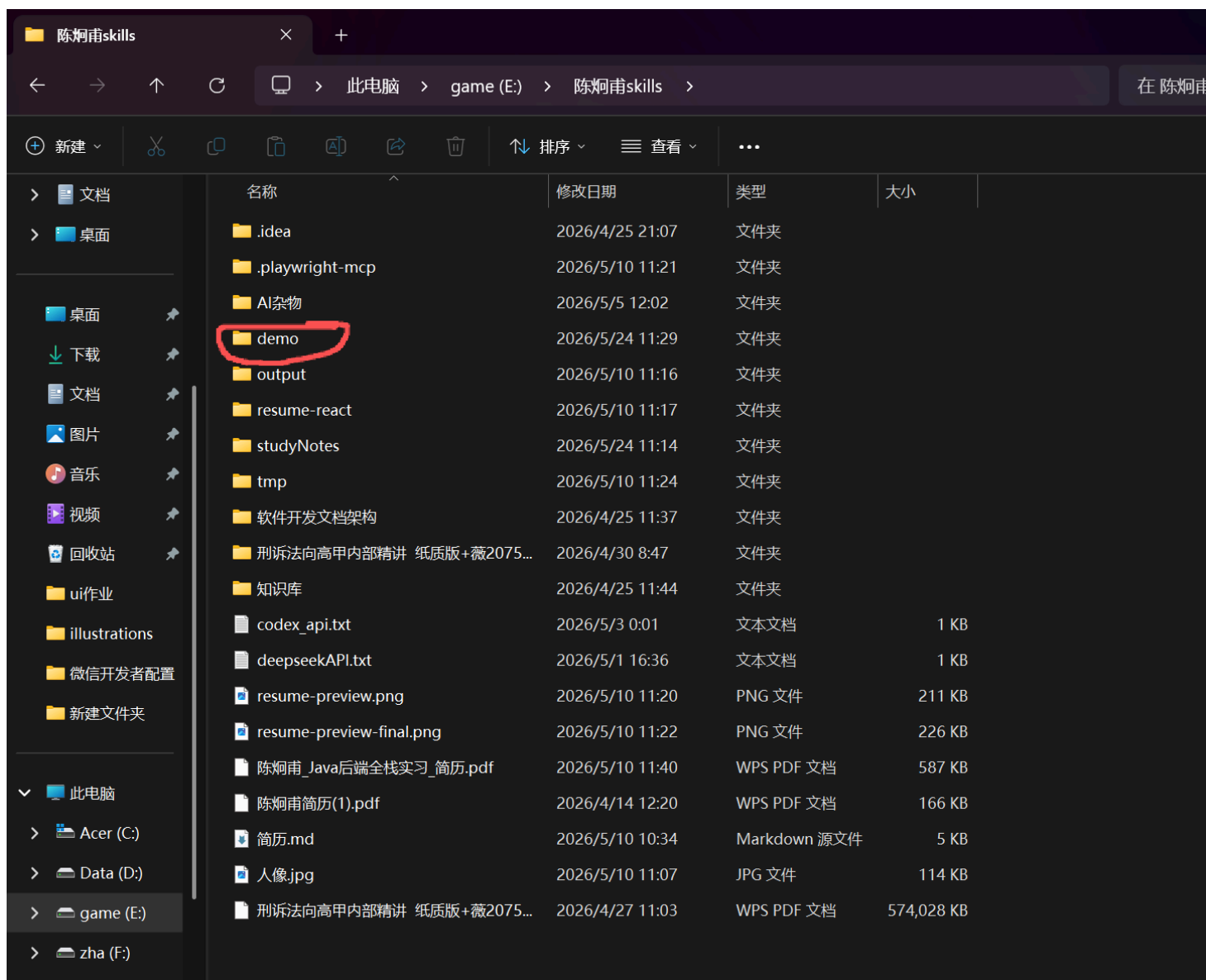
git clone 仓库地址(刚刚复制的)



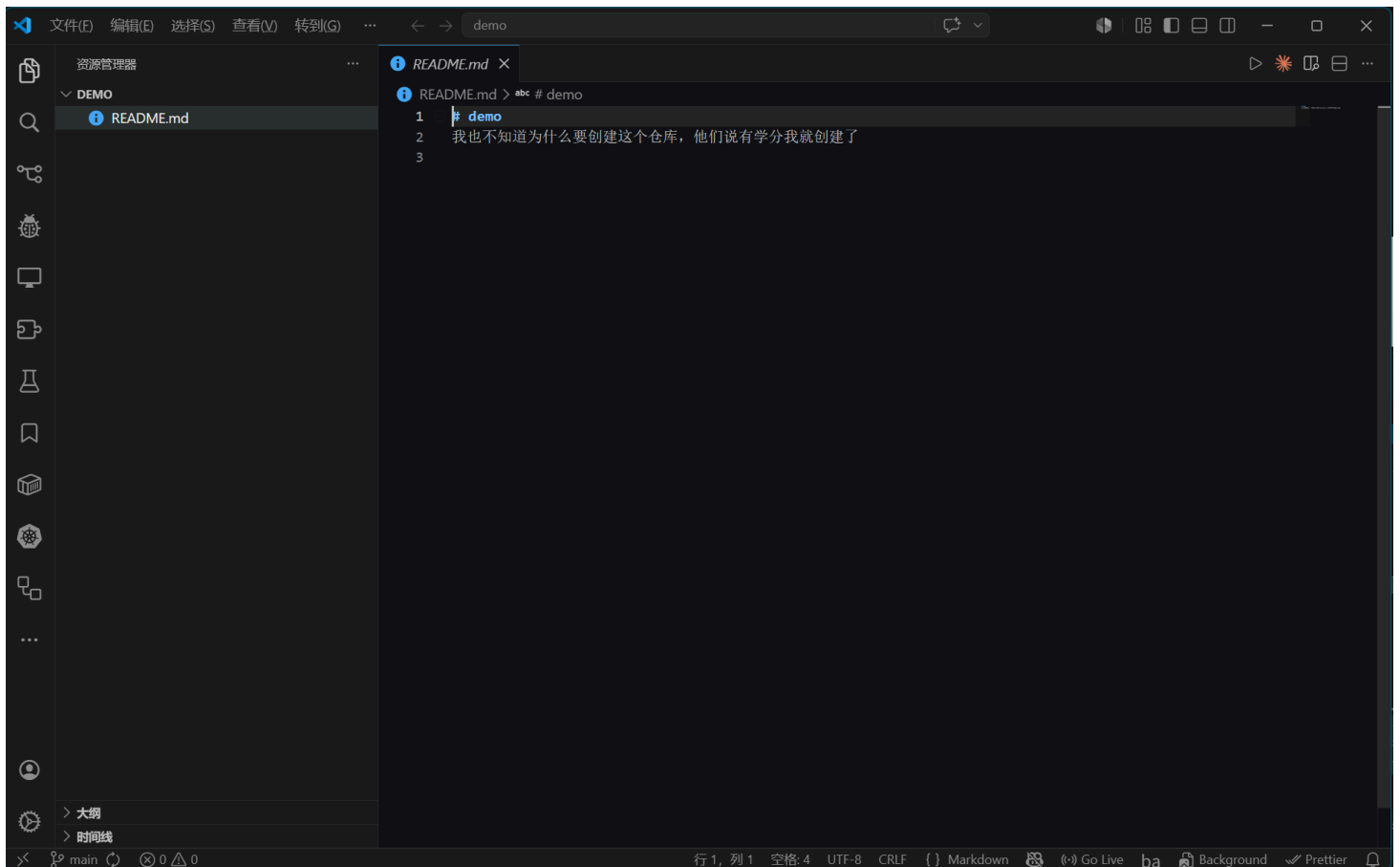
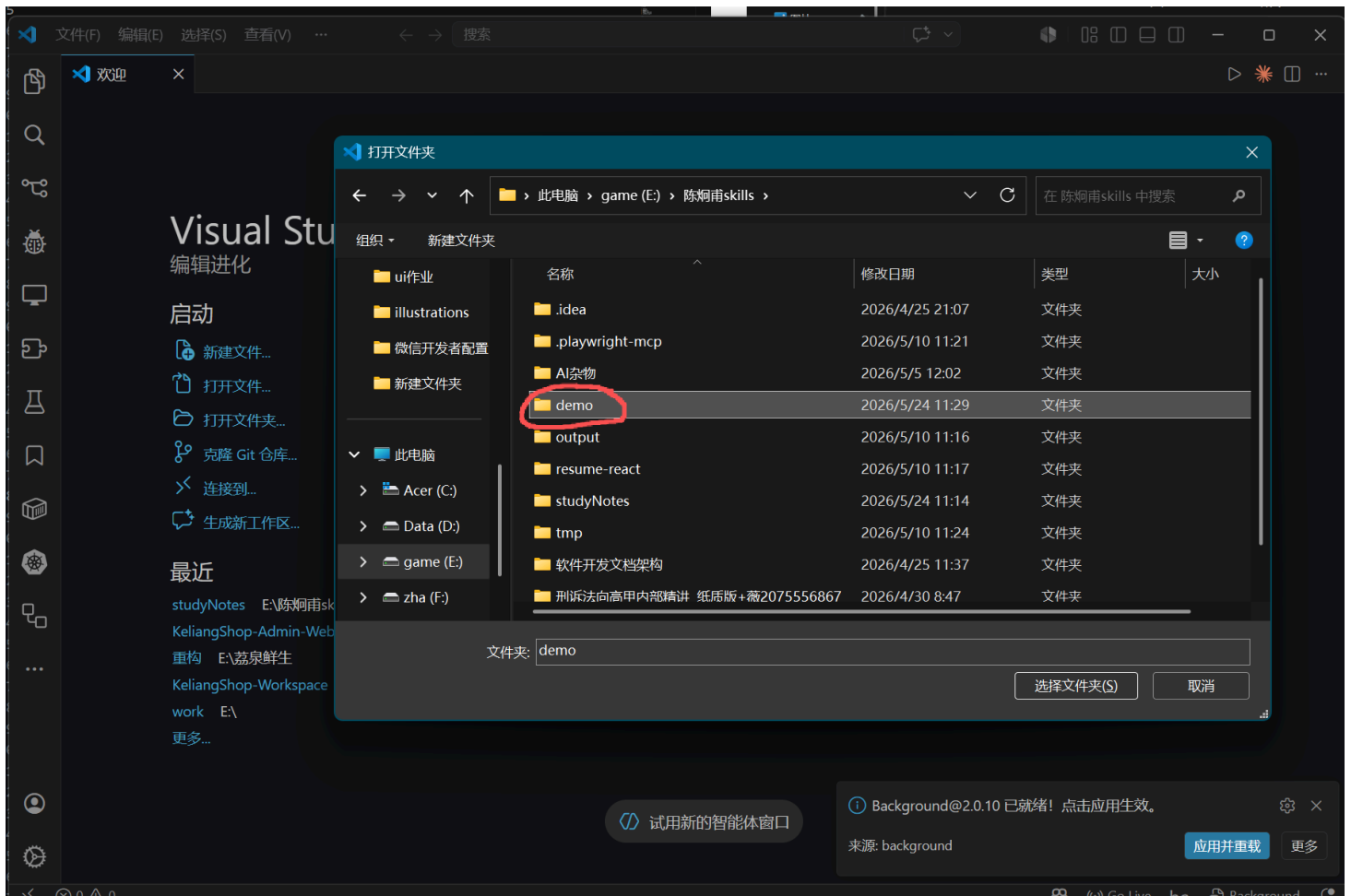
这样子就算成功了

```
C:\Windows\System32\cmd.exe × + v
E:\陈炯甫 skills>git clone git@github.com:chenqsq/demo.git
Cloning into 'demo'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.
E:\陈炯甫 skills>
```

在文件夹找到你的仓库



用vscode进入项目：



十三、Git 基础工作流（核心）

一、Git 基础命令

功能	命令	作用
查看版本	git --version	查看 Git 是否安装成功
初始化仓库	git init	创建 Git 仓库
克隆仓库	git clone 仓库地址	下载远程仓库
查看状态	git status	查看文件修改状态
添加全部文件	git add .	添加所有修改
提交代码	git commit -m "说明"	提交版本
推送代码	git push	上传代码到 GitHub
拉取代码	git pull	拉取远程最新代码
查看历史	git log --oneline	查看提交记录

二、Git 分支命令（重点）

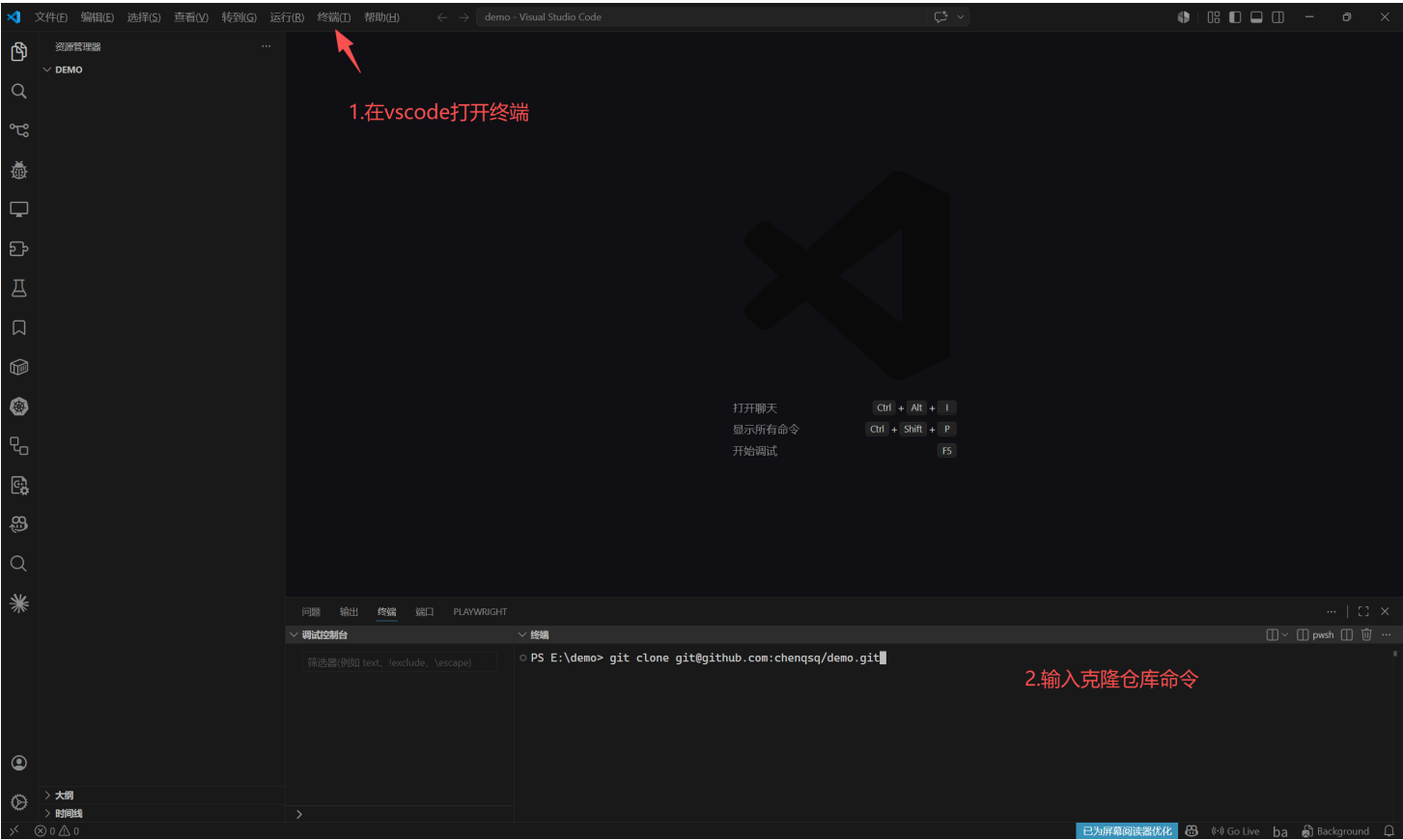
功能	命令	作用
查看分支	git branch	查看本地分支
查看远程分支	git branch -r	查看远程分支
查看所有分支	git branch -a	查看本地+远程
创建分支	git branch dev	创建 dev 分支
切换分支	git checkout dev	切换到 dev
创建并切换	git checkout -b dev	创建并切换分支

功能	命令	作用
新版切换分支	git switch dev	切换分支（新版推荐）
创建并切换（新版）	git switch -c dev	创建并切换
删除分支	git branch -d dev	删除分支
强制删除	git branch -D dev	强制删除分支
合并分支	git merge dev	合并 dev 到当前分支

三、最标准的企业开发流程

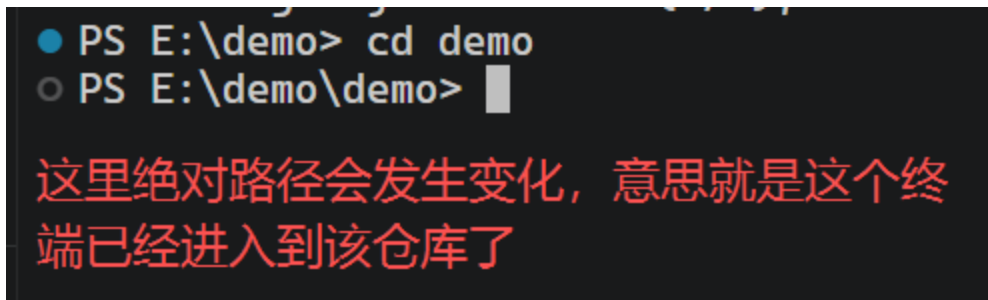
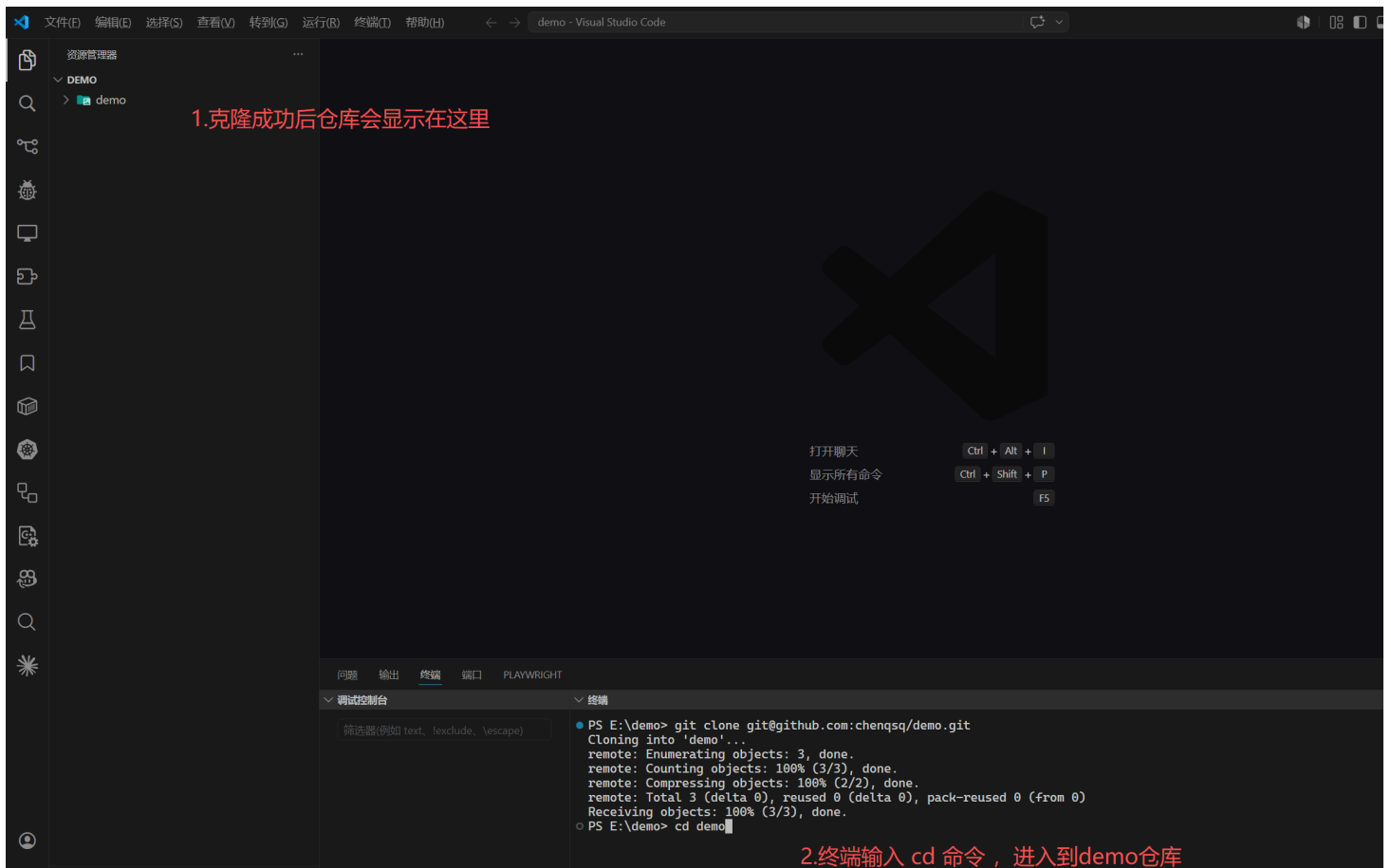
1. 克隆项目

git clone 仓库地址



2. 用终端进入项目/用vscode的打开文件夹(教程上面的步骤有)

cd 项目名



3. 在仓库查看当前分支

```
git branch
```

```
● PS E:\demo\demo> git branch
  * main
○ PS E:\demo\demo>
```

*号表示当前所在的分支

4. 创建功能分支

例如开发登录功能,输入这个命令后可以切换到该功能分支:

```
git checkout -b 分支名(英文)
```

或者新版:

```
git switch -c 分支名(英文)
```

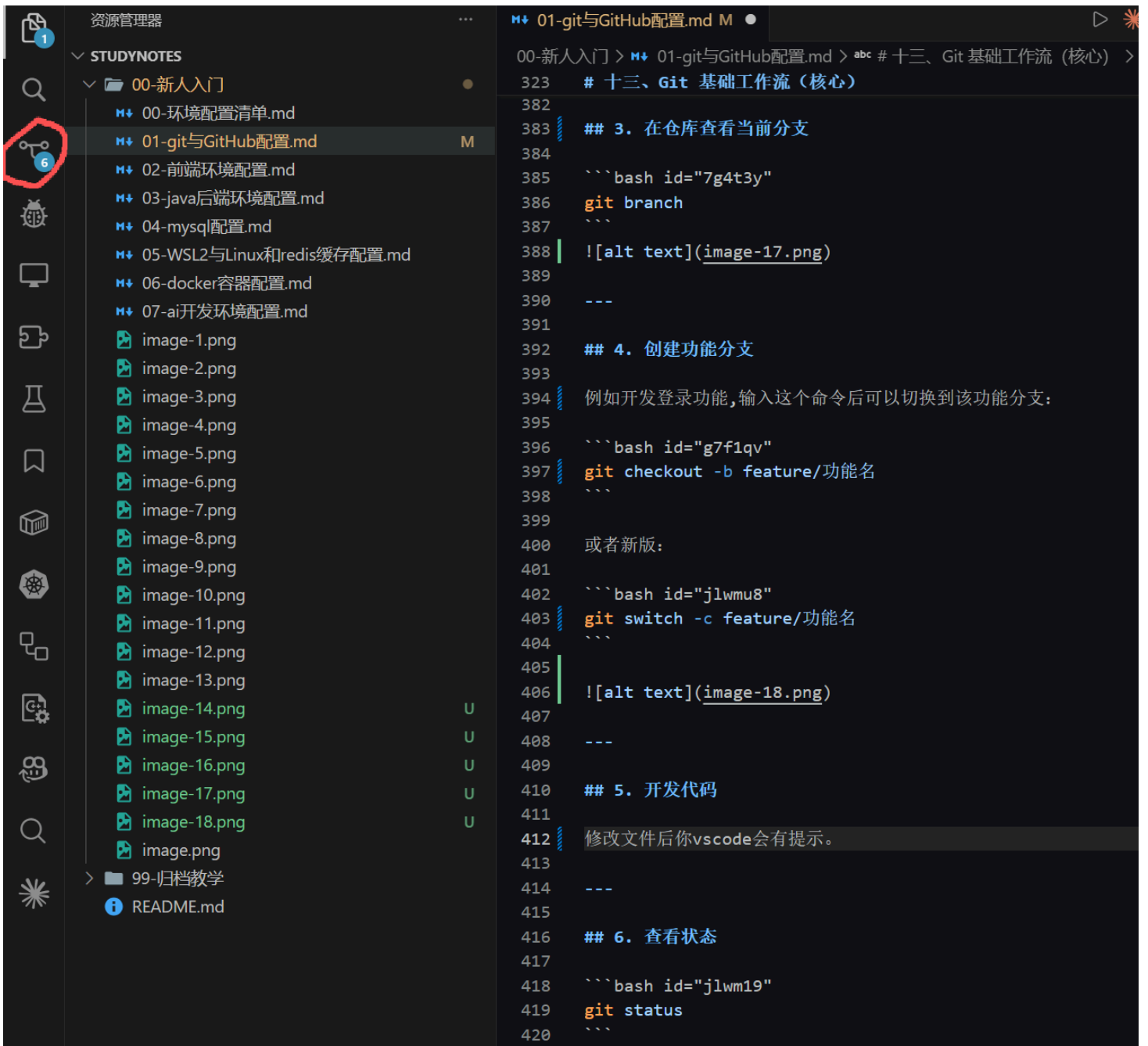
```
● PS E:\demo\demo> git checkout -b feature/demo
Switched to a new branch 'feature/demo'
● PS E:\demo\demo> git branch
  * feature/demo
   main
○ PS E:\demo\demo>
```

这个命令新建好后会自动切换到feature/demo分支

git branch命令查看当前分支确认切换成功

5. 开发代码

修改文件后你vscode会有提示



6. 暂存所有改动的代码/文件

```
git add .
```

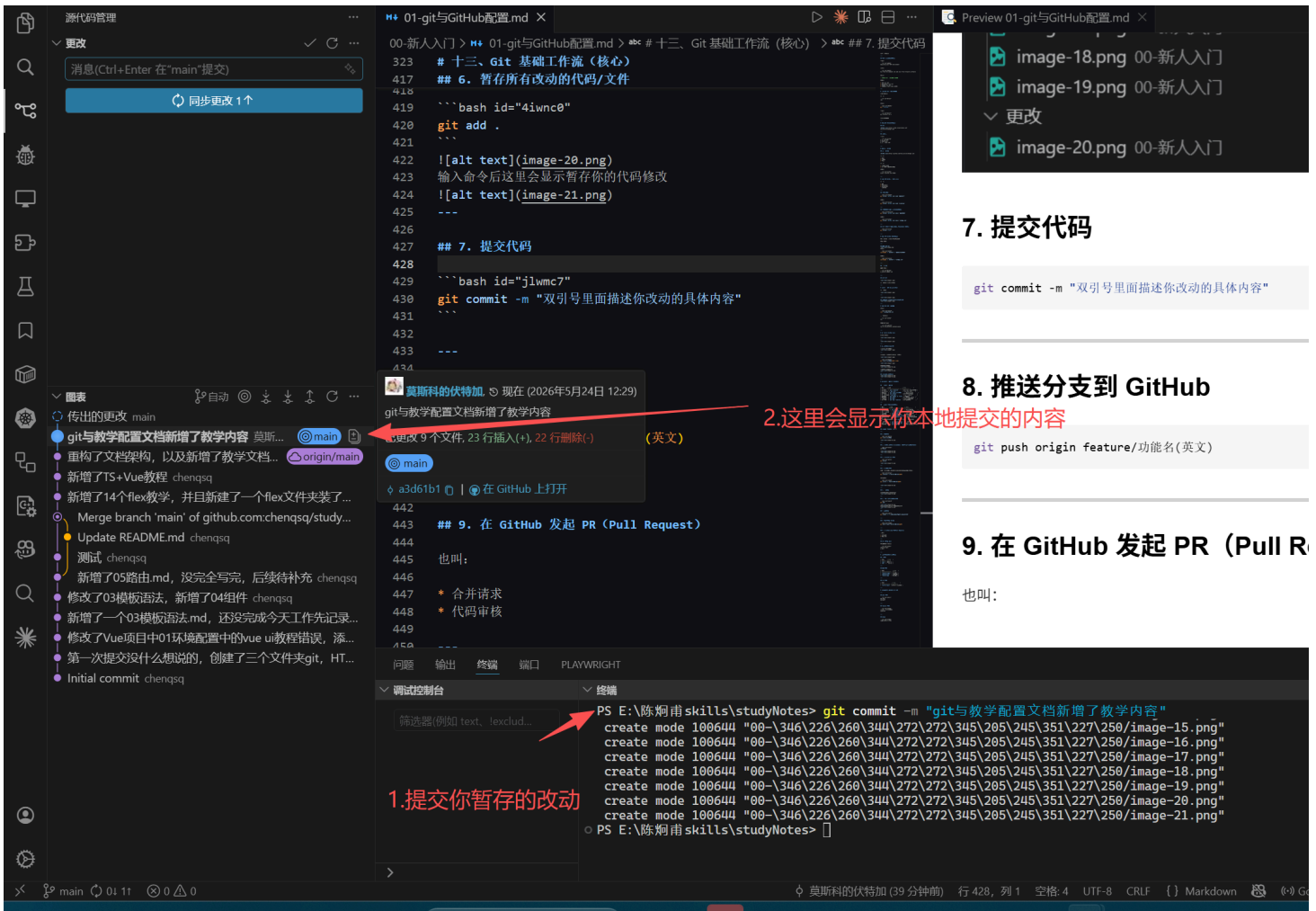
```
• PS E:\陈炯甫 skills\studyNotes> git add .  
❖ PS E:\陈炯甫 skills\studyNotes>
```

输入命令后这里会显示暂存你的代码修改



7. 提交代码

```
git commit -m "双引号里面描述你改动的具体内容"
```



8. 把你当前要推送的分支，推送分支到 GitHub 远程仓库

git push origin 分支名(英文)

```
终端
PS E:\demo> cd demo
PS E:\demo\demo> git push origin feature/demo
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 20 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 476 bytes | 238.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature/demo' on GitHub by visiting:
remote:   https://github.com/chenqsq/demo/pull/new/feature/demo
remote:
To github.com:chenqsq/demo.git
 * [new branch]      feature/demo -> feature/demo
💡 PS E:\demo\demo> █
```

这边推送了feature/demo分支上去GitHub仓库了

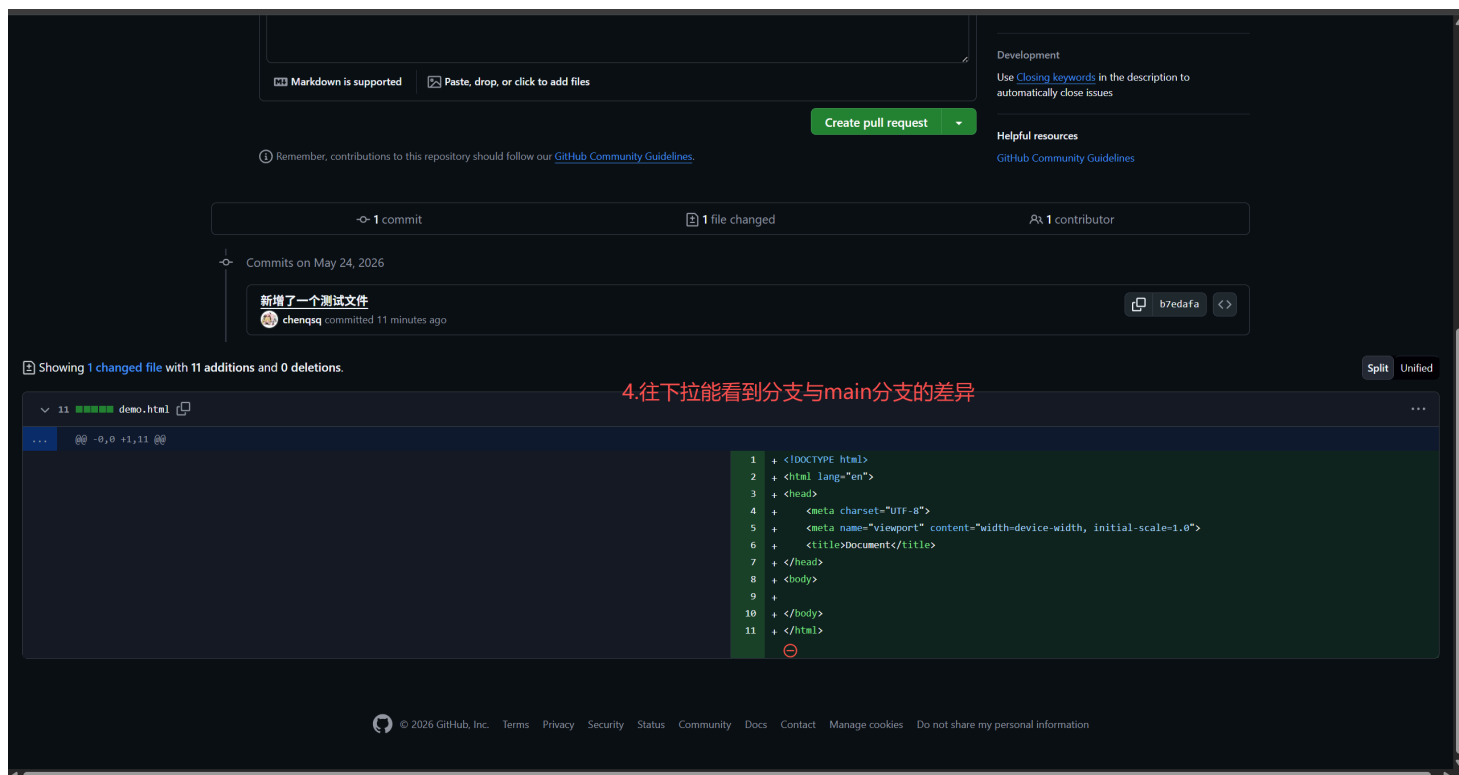
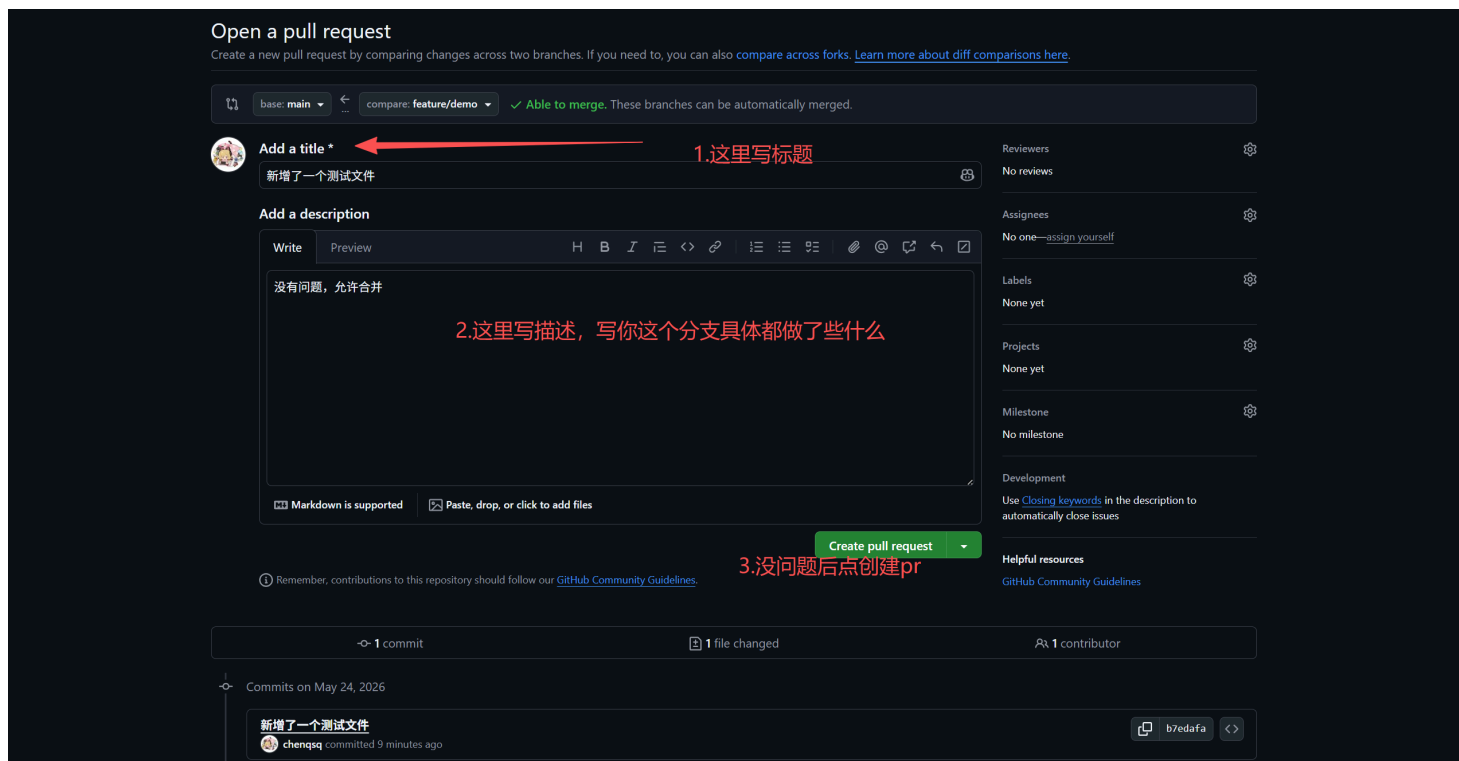
9. 在 GitHub 发起 PR (Pull Request)

也叫：

- 合并请求
- 代码审核

The screenshot shows the GitHub repository 'demo' by user 'chenqsq'. The repository has 2 branches: 'main' and 'feature/demo'. The 'feature/demo' branch is highlighted with a red box and a red arrow pointing to it, with the text '1. 这里能看到刚刚推送上来的分支' (1. You can see the branch just pushed up here). Another red box highlights the 'Compare & pull request' button, with a red arrow pointing to it and the text '2. 这里点击创建pr, 一般情况下都是由一个项目负责人负责审查你的分支, 没冲突且代码没问题后会合并到main分支' (2. Click here to create a PR, generally it's the project manager who reviews your branch, and if there's no conflict and the code is fine, it will be merged to the main branch).

我也不知道为什么要创建这个仓库，他们说有学分我就创建了



10. 合并到 main

项目负责人审核后：

feature/login

→ main

新增了一个测试文件 #1

Ready to merge Code

Open chengsq wants to merge 1 commit into main from feature/demo

Conversation 0 Commits 1 Checks 0 Files changed 1

chengsq commented now

没有问题，允许合并

新增了一个测试文件 b7edafa

No conflicts with base branch
Merging can be performed automatically.

Merge pull request You can also merge this with the command line. [View command line instructions.](#)

Still in progress? Convert to draft

Add a comment

Write Preview

Add your comment here...

Markdown is supported Paste, drop, or click to add files

Close pull request Comment

Remember, contributions to this repository should follow our [GitHub Community Guidelines](#)

Reviewers
No reviews
Still in progress? [Convert to draft](#)

Assignees
No one—[assign yourself](#)

Labels
None yet

Projects
None yet

Milestone
No milestone

Development
Successfully merging this pull request may close these issues.
None yet

Notifications
Unsubscribe
You're receiving notifications because you authored the thread.

1 participant

审核人员会在这里进行审核没问题后就点这个通过

新增了一个测试文件 #1

Merged chengsq merged 1 commit into main from feature/demo now

Conversation 0 Commits 1 Checks 0 Files changed 1

chengsq commented 1 hour ago

没有问题，允许合并

新增了一个测试文件 b7edafa

chengsq merged commit b3e3d3c into main now

Revert

Pull request successfully merged and closed
You're all set—the `feature/demo` branch can be safely deleted.

Delete branch

Add a comment

Write Preview

Add your comment here...

Markdown is supported Paste, drop, or click to add files

Comment

Remember, contributions to this repository should follow our [GitHub Community Guidelines](#)

Reviewers
No reviews
Still in progress? [Convert to draft](#)

Assignees
No one—[assign yourself](#)

Labels
None yet

Projects
None yet

Milestone
No milestone

Development
Successfully merging this pull request may close these issues.
None yet

Notifications
Unsubscribe
You're receiving notifications because you modified the open/close state.

1 participant

合并成功后如果该分支不会继续使用，则进行删除

四、分支规范（非常重要）

主分支

分支	作用
main	生产环境
dev	开发环境

功能分支（用feature/作为前缀）

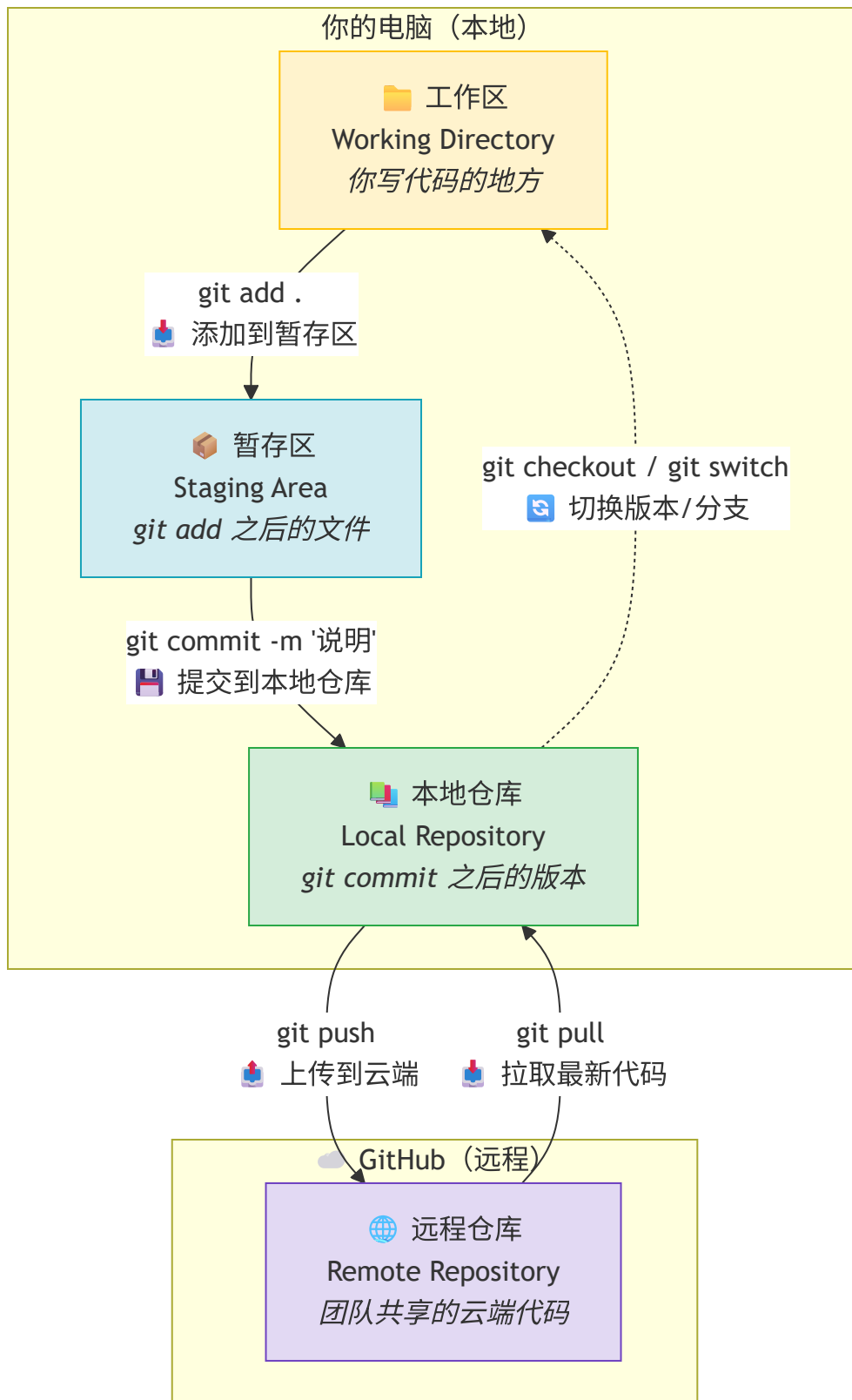
分支	作用
feature/login	登录功能
feature/user	用户模块
feature/pay	支付功能

修复分支

分支	作用
hotfix/login	紧急修复登录问题

七、新人必须理解的 Git 思维

思维总览图（一图看懂 Git 全链路）



核心思维一：三区模型（最重要的概念）

新人最容易迷惑的就是 Git 的「三个区域」：

区域	命令	说明
工作区	写代码	你正在修改的文件
暂存区	git add	标记哪些文件要纳入下一次提交
版本库	git commit	生成一个版本快照，永久保存

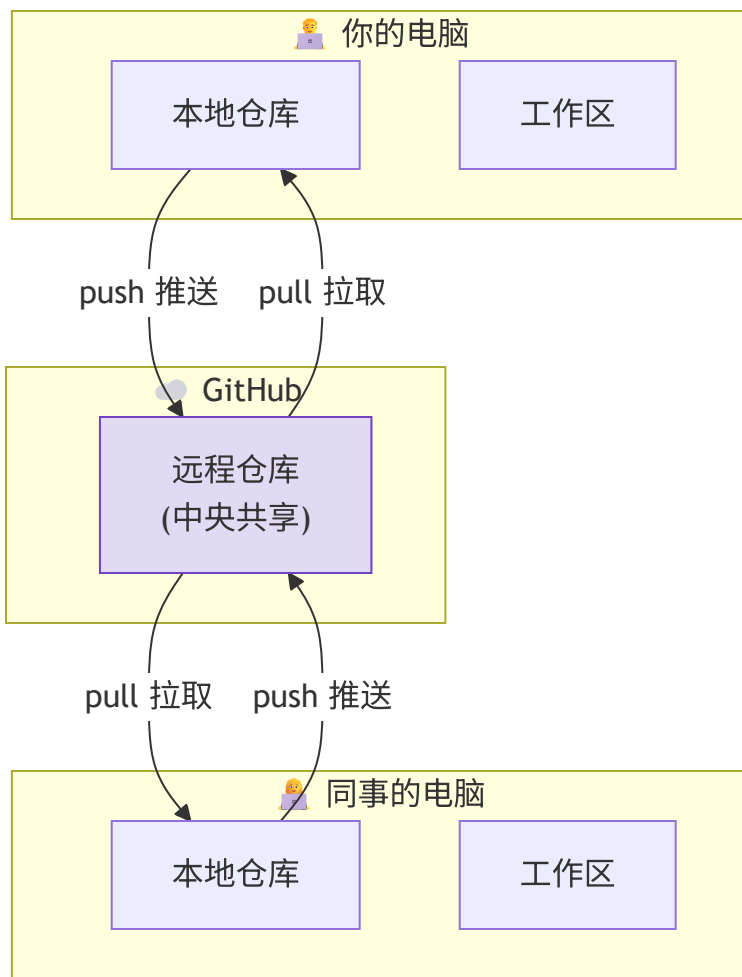


⚠️ **关键理解：** git add 和 git commit 是两步，缺一不可！

- 只 git add 不 git commit = 文件标记了但还没生成版本记录
- 直接 git commit 不 git add = 没有文件被暂存，commit 不会包含任何改动

核心思维二：分布式协作（本地 ↔ 远程）

Git 是**分布式**版本控制，每个人都有完整的仓库副本。

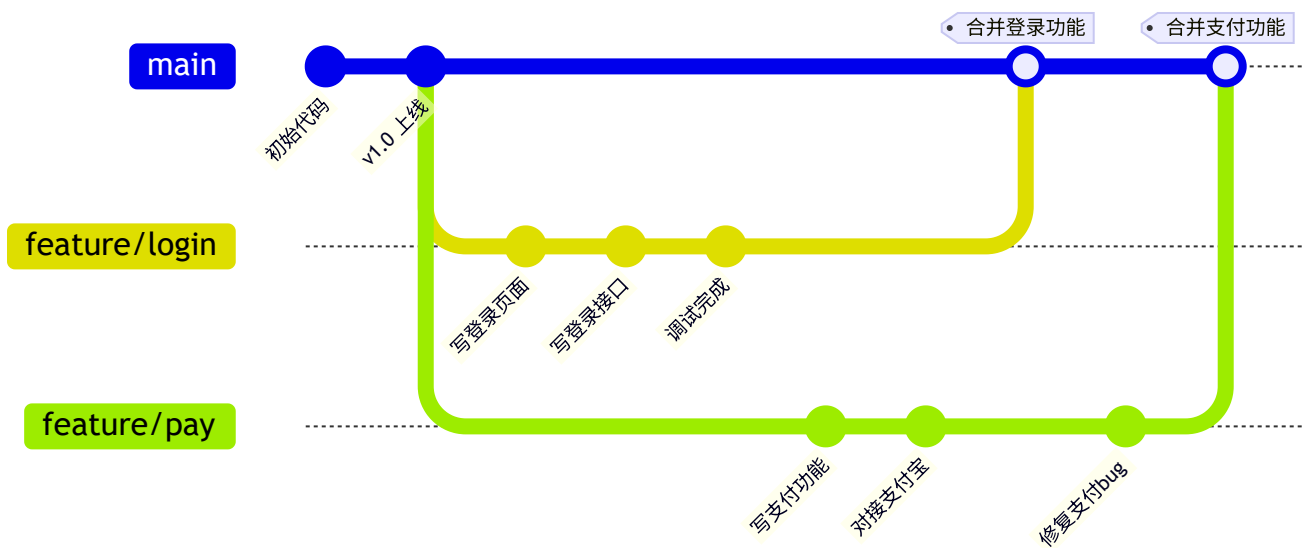


一句话总结：

- push = 把你的代码**分享**给团队
- pull = 把团队的代码**同步**到本地

核心思维三：分支开发（互不干扰的平行宇宙）

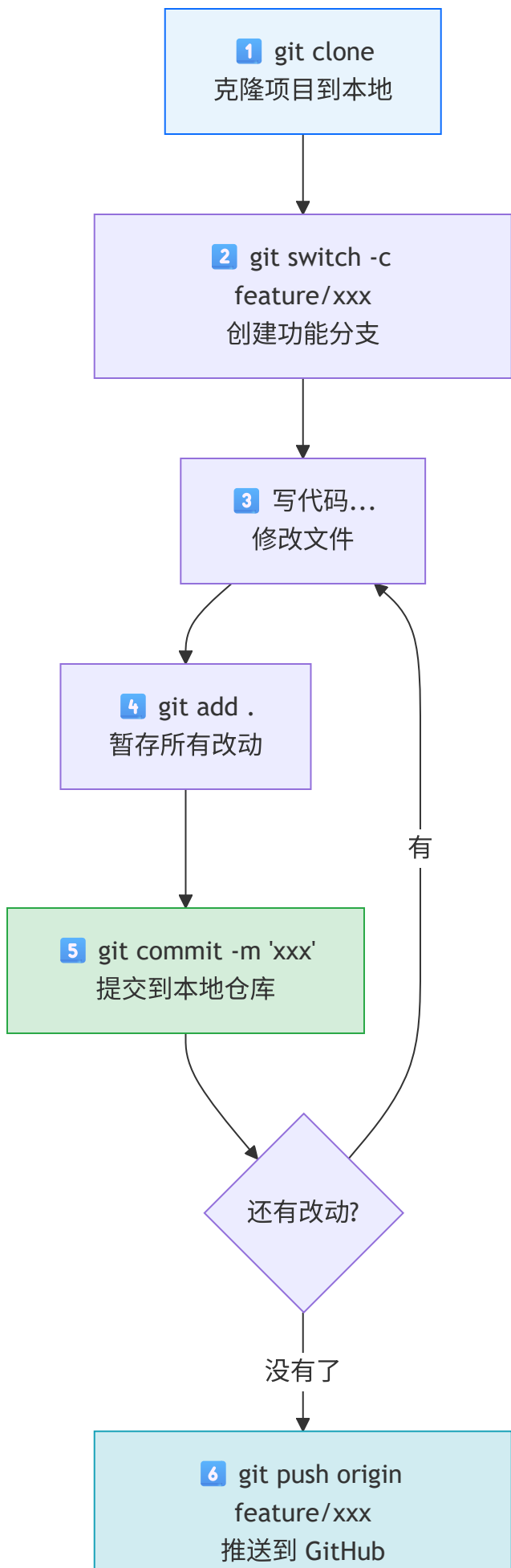
分支让你可以**安全地实验**，不影响主代码。

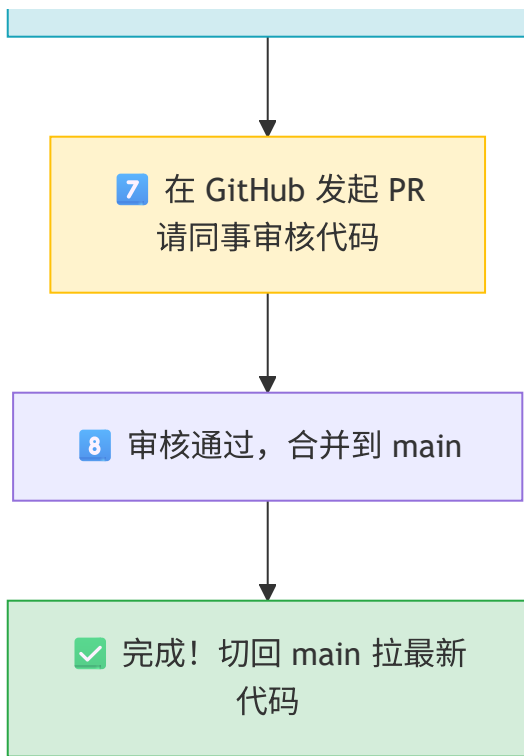


分支思维核心：

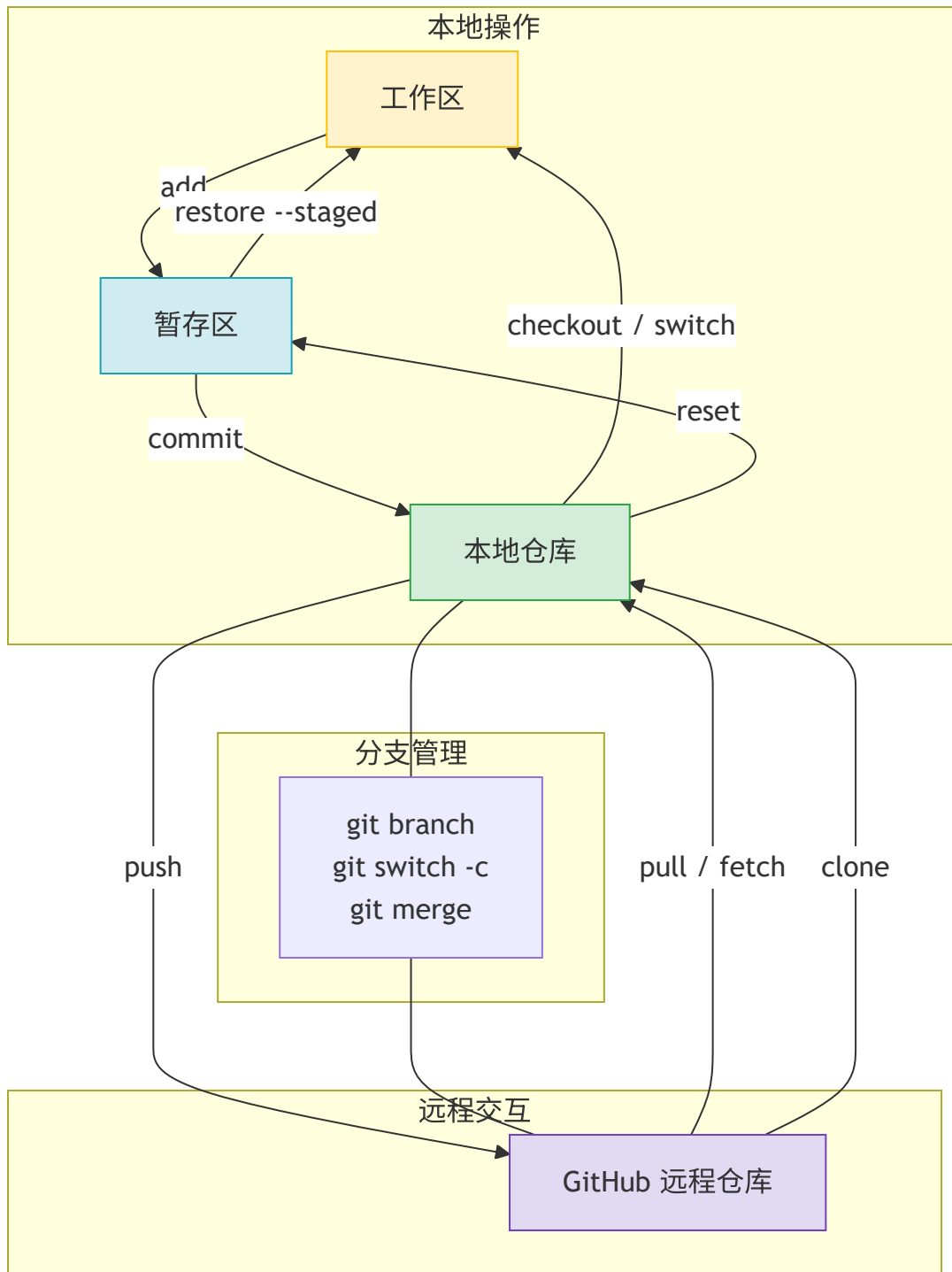
- **main** 分支 = 稳定版本（永远可运行）
- **feature/xxx** 分支 = 开发新功能（坏了大不了删掉重来）
- 开发完 → 测试通过 → 合并回 **main**

核心思维四：完整开发链路（每天做的事）





一张图记住所有命令的关系



新人常见误区总结

误区	正确理解
✗ git add 就是保存了	✓ git add 只是标记文件，git commit 才是真正生成版本
✗ git push 后别人立刻能看到	✓ 还需要在 GitHub 上发起 PR 并合并
✗ 直接在 main 分支上写代码	✓ 必须创建 feature/xxx 分支开发
✗ 改坏了代码就完了	✓ git checkout 可以回退到任意历史版本
✗ Git 和 GitHub 是一回事	✓ Git 是工具，GitHub 是网站平台